

SWIFT

THE EXECUTION PLAYBOOK

Companion to the Master Plan — Step-by-Step Build Operations

Copy-paste Claude Code prompts · Acceptance criteria · CLAUDE.md · Sprint schedule

Use with: Swift — The Master Plan (strategy + rules).
This volume is purely operational: it tells you exactly what to type, in what order,
and how to know each step is actually done.

Westbridge Inc · Prepared June 2026 · Confidential

0. How to Run This Build

0.1 The golden workflow (every single step)

#	Action
1	Open a FRESH Claude Code session in the repo root (fresh context per step — don't let sessions sprawl)
2	Paste the step's prompt from this playbook. The prompt always demands a PLAN first.
3	Read the plan. Approve, or correct it before any code is written.
4	Let it write code + tests for this one concern only. If it starts drifting into other steps, stop it.
5	Run: typecheck, lint, tests. Iterate until green. Never accept 'mostly passing.'
6	Review the diff YOURSELF — every file. You don't need to understand every line; you need to confirm it only touched what the step covers.
7	Commit with a clear message. Then — and only then — next step.

The five rules Claude Code must never break (also in CLAUDE.md)

1. Money, auth, billing, verification decisions = deterministic code. Never an AI/LLM call.
2. Never hold order money. Cash is recorded only. You are SaaS, not a money handler.
3. No PII, documents, or payment data to any external AI. Claude API = language layer only.
4. Every external service behind a swappable interface (PaymentProvider, KycProvider, Maps, Notifications, BankingProvider).
5. One concern per step. Plan → code → green → commit → review → next.

0.2 Model routing during the build

Work	Model	Why
Steps 0–2, 8, 10, PR triage, any multi-file refactor	Fable 5 (in the included window; credits after Jun 22)	Long-horizon, multi-step, failure-path-heavy — where its lead is largest
Steps 3–7, 9, 11–14 routine portions	Opus 4.8	Included in plan; near-equal on well-scoped tasks; half cost
Boilerplate, copy tweaks, simple screens	Sonnet	Fast and cheap; quality sufficient

Switch with /model inside Claude Code. Start Max 5x; upgrade to 20x only if you consistently hit limits mid-sprint.

1. CLAUDE.md — put this file in the repo root first

Claude Code reads CLAUDE.md automatically every session. This is how every session knows the locked model without you re-explaining. Create it before Step 0. Copy the text below verbatim into CLAUDE.md:

Paste this prompt:

```
# Swift — Claude Code Instructions

## What this is
Swift: Caribbean super app (Guyana launch). Two-sided marketplace.
Vendors + Movers pay flat WEEKLY SUBSCRIPTIONS and keep 100% of earnings.
Customers pay nothing to the platform. Revenue = subscriptions + ads.
V1 is CASH-ONLY: no order payment processing, no wallet, no commission.
Full spec: docs/swift-master-plan.pdf (Section 18 lists superseded decisions).

## Hard rules (never break)
1. Money, auth, billing, verification = deterministic code. NEVER an AI call.
2. Never hold or process order money. Cash is recorded only.
3. No PII / documents / payment data to any external AI service.
4. All external services behind swappable interfaces:
   PaymentProvider, KycProvider, Maps, Notifications, BankingProvider.
5. One concern per session. Propose a plan BEFORE writing code.

## Locked domain model
Roles: CUSTOMER, MOVER (rider and/or driver), VENDOR.
VendorType: RESTAURANT | SUPERMARKET | STORE | SERVICE (exactly 4).
Listing: one model for goods AND services (services carry bookingConfig:
  duration, slots). fulfillment: DELIVERY | APPOINTMENT | PICKUP.
Order states: placed -> accepted -> preparing -> ready -> picked_up
  -> delivered | cancelled. Immutable order event log.
Trust levels: L1 phone-verified (OTP, mandatory), L2 ID-verified
  (required for orders >= USD $50 equivalent), L3 earned/trusted.
Cash rules: rider float (rider pays vendor at pickup, collects from
  customer); GOLDEN RULE: payment before handover; under-$50 company
  guarantee with GPS/photo claim flow; strike system on customers.
Subscriptions: weekly, tiered (rider 12000 GYD, small vendor 20000,
  large 30000, franchise per store, catalogue-size tiers above).
  Auto-suspend on non-payment. Prepaid balance path for cash vendors.
CountryConfig wraps currency, thresholds, tiers, document checklists.

## Engineering standards
- TypeScript strict. Zod (or equiv) validation on every route.
- Tests required for every business rule; failure paths first.
- Idempotency keys on all billing calls; audit log on money events.
- PostGIS for mover geolocation; indexes on hot queries.
- No secrets in code. Env only.
```

Also drop swift-master-plan.pdf (and this playbook) into docs/ in the repo so sessions can reference them.

2. The Sixteen Steps

Each step below is one (sometimes two) Claude Code sessions. Prompts are written to be pasted as-is. Where a prompt says , paste the relevant findings from Step 0's report.

STEP 0 — Orientation (read-only)

Goal: Know exactly what exists before changing anything. No code changes in this step.

Paste this prompt:

```
Read CLAUDE.md first. This is a READ-ONLY orientation session - propose no code changes. Do the following and produce a single ORIENTATION.md report:
1. Open package.json, turbo.json, and every workspace under apps/* and packages/*. Report: frameworks + versions, the database + ORM, whether PostGIS is enabled, how the backend (Fastify?) is structured.
2. List every existing Prisma/DB model with its fields. Flag which of these exist and how they differ from CLAUDE.md's locked domain model: User, Vendor (VendorType values?), Mover, Item/Listing, Order (states?), Subscription, CountryConfig.
3. List the 12 open PRs by title; for each, say in one line what it touches and whether it conflicts with the locked model.
4. List anything that looks like payment-processor or wallet code (this is slated for removal/feature-flagging per the master plan Section 18).
5. End with: top 10 risks you see, ranked.
```

Acceptance criteria (all must be true before commit):

- ORIENTATION.md exists and answers all five points concretely (no 'it appears that' hedging on checkable facts)
- Every existing model is listed with its actual fields
- Each open PR has a one-line verdict: merge-candidate / close / rework
- Zero code changes were made

Done when

You can read ORIENTATION.md and know exactly what you have. Decisions on the 12 PRs are made (by you) before Step 1.

STEP 1 — Stabilise & secure (do first, always)

Goal: The repo's known security issues fixed, hot-path indexes added, validation everywhere. Nothing new gets built on an insecure base.

Paste this prompt:

```
Read CLAUDE.md and ORIENTATION.md. This session is stabilisation only - no new features. Propose a plan first, then:  
1. Fix the known critical security issues (including the wallet race condition and the Socket.IO auth bypass identified in earlier audits - verify each against current code, fix, and add a regression test).  
2. Audit every path touching money, auth, billing, verification. Confirm none uses an AI call for a decision. Remove any that do.  
3. Add DB indexes: mover geolocation (geospatial), mover online status, subscription dueDate, order status (+ any hot query ORIENTATION found).  
4. Add zod validation on every API route; reject malformed input early. Rate-limit auth endpoints.  
5. Confirm all secrets load from env; complete .env.example; remove any hardcoded keys.  
6. Output SECURITY-FINDINGS.md: fixed items, and anything that still needs a human/professional review.
```

Acceptance criteria (all must be true before commit):

- All previously flagged critical issues fixed each with a regression test
- Typecheck + lint + full test suite green
- SECURITY-FINDINGS.md lists what was fixed and what remains for human review
- No route accepts unvalidated input

Test hardest:

- Concurrent requests against any money/balance-adjacent endpoint (the race-condition class)
- Auth bypass attempts on Socket.IO connections and role-crossing API calls (customer hitting vendor/admin endpoints)

Done when

The suite is green, the findings file exists, and you've read it. This step is the foundation - do not rush it.

STEP 2 — Schema & multi-country (align to the locked model)

Goal: The database schema matches the Master Plan exactly. This is the highest-leverage step of the sprint.

Paste this prompt:

```
Read CLAUDE.md (locked domain model) and the master plan in docs/.
Propose the full migration plan FIRST including what existing data/tables
are affected. Then implement:
1. CountryConfig: code, currency, ID-gate threshold (USD-equivalent),
  subscription fee tiers, required-document checklists per role,
  active providers. Seed Guyana (GYD, $50-equivalent gate, tiers
  12000/20000/30000).
2. VendorType enum -> exactly RESTAURANT, SUPERMARKET, STORE, SERVICE.
3. Listing model: name, price, description, images, options, available
  (vendor toggle), optional bookingConfig (duration, slots) for services.
  fulfillment: DELIVERY | APPOINTMENT | PICKUP.
4. Order: states placed->accepted->preparing->ready->picked_up->
  delivered|cancelled; immutable OrderEvent log; totals incl. delivery fee.
5. Subscription: payer (vendor or mover), tier, weekly amount, currency,
  dueDate, status, auto-suspend flag. PrepaidBalance for cash payers.
6. Trust: trustLevel L1/L2/L3 on customer accounts; Strike model
  (customer, phone, address counters); VerificationDocument model with
  states pending->approved->rejected->expired.
7. QR: vendor qrSlug (unique, stable) for deep links.
8. Remove or feature-flag dormant wallet/payment-processing models per
  master plan Section 18 (keep the ledger-ready shape for Part C later).
Generate migrations; do NOT run against any non-dev database.
```

Acceptance criteria (all must be true before commit):

- Schema compiles; migrations generated and reviewed by you before running anywhere
- Every element of the locked domain model exists exactly as specified (4 vendor types, 3 trust levels, order states, QR slug)
- Guyana CountryConfig seeded; thresholds and tiers read from config, not hardcoded
- Old payment-processing models removed or flagged with a comment pointing at master plan §18

Test hardest:

- Write a seed script + a smoke test that creates one vendor of each type, one listing of each fulfillment kind, and walks an order through every state

Done when

Schema mirrors the Master Plan one-to-one and the smoke test passes. README updated to describe the subscription model (not the old commission/Uber framing).

STEP 3 — Auth & accounts

Goal: Signup/login for all roles with phone OTP, multi-role users, hardened sessions.

Paste this prompt:

```
Read CLAUDE.md. Plan first. Implement:
1. Signup/login for CUSTOMER, MOVER, VENDOR. Phone + SMS OTP (Twilio
   Verify behind the Notifications interface) and email+password.
   OTP at signup is MANDATORY for customers (trust Level 1).
2. Multi-role per user (a Mover can be a Customer).
3. Country picked at signup (from live CountryConfigs); role-specific
   onboarding routing stubs (verification comes in Step 4).
4. Secure sessions/tokens, refresh, password reset, rate limiting,
   lockout on repeated failures.
5. NO AI calls anywhere in auth. Tests for every flow incl. failures.
```

Acceptance criteria (all must be true before commit):

- All roles can sign up and log in; OTP verified accounts are marked Level 1
- Role-crossing authorization tests pass (customer cannot call vendor/admin endpoints)
- Rate limits and lockouts proven by tests
- Suite green

Test hardest:

- Token theft/replay attempts, expired-token paths, OTP brute-force

Done when

A user can sign up as each role on a seeded Guyana config, and the auth test file reads like an attack checklist.

STEP 4 — Verification behind KycProvider

Goal: Identity API + manual review queue + expiry automation, per the hybrid model.

Paste this prompt:

```
Read CLAUDE.md + master plan Section 6. Plan first. Implement:
1. KycProvider interface (verifyIdentity, verifyDocument, status) with a
  stub/sandbox adapter now (Sumsb-class later - keep it swappable).
2. Required-document checklists come from CountryConfig per role:
  movers (ID, driver's licence, vehicle reg+insurance; taxi adds
  hire-car permit + vehicle/plate photo), restaurants (owner ID,
  business registration, food handler cert), stores/supermarkets
  (owner ID, business registration), services (ID minimum).
3. VerificationDocument lifecycle: pending->approved->rejected->expired;
  OCR'd expiry dates; reminder 30 days out; auto-suspend listings on
  critical-document lapse.
4. Manual admin review queue: list pending, view docs, approve/reject
  with reason, applicant notified to fix and resubmit.
5. Level 2 customer verification flow: ID + selfie via the interface ->
  permanent Verified flag.
6. Gate: no listing / no orders / no rides until role verification passes.
7. Store only verification RESULT + reference token. Raw docs encrypted
  only if retention is required. Nothing to logs. Nothing to AI.
```

Acceptance criteria (all must be true before commit):

- Checklists drive from config; adding a country = config, not code
- Full lifecycle tested incl. expiry auto-suspend
- Queue usable end-to-end with notifications
- Gating enforced by tests (unverified vendor cannot list)
- Suite green

Test hardest:

- Document-state edge cases: resubmission after rejection, expiry during pending review

Done when

A fake vendor can be walked from signup through manual approval to active, and an expired licence suspends them automatically.

STEP 5 — Subscription billing engine (the revenue engine)

Goal: Weekly tokenised billing + prepaid balance + auto-suspend. The one place V1 touches money - your own fee.

Paste this prompt:

```
Read CLAUDE.md + master plan Section 10. Plan first. Implement:
1. PaymentProvider interface (tokenizeCard, chargeToken, refund) with a
  sandbox adapter. Real adapter (PowerTranz/WiPay-class) slots in later;
  nothing outside the adapter may know which processor exists.
2. Weekly billing job: charge each active Subscription on dueDate via
  stored token OR deduct from PrepaidBalance for cash payers.
3. Failure -> retry policy -> AUTO-SUSPEND access until paid. Reinstate
  on payment. Notifications at each stage (reminder, failed, suspended,
  reinstated).
4. PrepaidBalance top-up recording (bank transfer / agent - manual
  confirm in admin for now).
5. Idempotency keys on every charge attempt; immutable BillingEvent
  audit log; deterministic code only.
6. Tier assignment from CountryConfig; catalogue-size tier recalculated
  from activeListingCount weekly (never from sales).
```

Acceptance criteria (all must be true before commit):

- Billing job is idempotent: running it twice cannot double-charge (proven by test)
- Suspend/reinstate proven by tests; suspended vendors vanish from customer browse
- Audit log captures every money event
- Suite green

Test hardest:

- Clock-edge cases (dueDate boundaries, retries across days), partial failures mid-batch, idempotency under concurrent job runs

Done when

You can simulate a week: vendors billed, one card fails -> suspended -> tops up -> reinstated, and the audit log tells the whole story.

STEP 6 — Catalogue

Goal: One listing system for everything sellable; CSV import; the availability toggle.

Paste this prompt:

```
Read CLAUDE.md. Plan first. Implement:
1. Listing CRUD for vendors: manual entry + CSV bulk import (template
  download, row-level error reporting on import).
2. available toggle per listing - instant effect on customer browse.
3. bookingConfig for SERVICE listings: duration, available slots,
  double-booking prevention at the data layer.
4. Image upload + storage behind an interface; size/type validation.
5. Category metadata drives browse/filtering ONLY - one code path for
  all four vendor types (test: same endpoints serve a burger and a
  haircut).
```

Acceptance criteria (all must be true before commit):

- CSV import handles a messy real file (bad rows reported, good rows imported)
- Toggle reflects in customer-facing queries immediately
- Double-booking on a service slot is impossible (concurrent-booking test)
- Suite green

Test hardest:

- Concurrent bookings on the same slot; huge CSVs; malformed images

Done when

A supermarket can upload 500 items by CSV, flip three to sold-out, and a barber can list a 30-min haircut with Tuesday slots - all through the same system.

STEP 7 — Cart, checkout & order lifecycle

Goal: Orders flow through the locked states with the trust/ID gates wired in. Cash totals always upfront.

Paste this prompt:

```
Read CLAUDE.md + master plan Sections 4-5. Plan first. Implement:
1. Cart (single + multi-vendor), checkout with fulfillment selection
   (delivery/pickup/appointment) and upfront totals: order + delivery fee.
2. Gate checks at placement: customer is L1 minimum; if total >= the
   CountryConfig ID-gate threshold, require L2 Verified - block with a
   clear upgrade path otherwise. Risk flag: new account + high value +
   odd hours -> vendor sees call-to-confirm prompt.
3. Order state machine exactly: placed->accepted->preparing->ready->
   picked_up->delivered|cancelled. Only valid transitions possible;
   every transition appends an immutable OrderEvent.
4. Appointment orders book the slot at acceptance; pickup orders skip
   dispatch.
5. Receipts (recorded, since cash); customer order history; reorder.
```

Acceptance criteria (all must be true before commit):

- Invalid state transitions are impossible (exhaustive transition test)
- ID gate proven: unverified L1 blocked at threshold, Verified L2 flows
- Multi-vendor cart splits into per-vendor orders correctly
- Suite green

Test hardest:

- State machine under concurrency (vendor accepting while customer cancels), threshold boundary values, multi-vendor edge cases

Done when

An order can be walked through every legal path - and pushed at every illegal one and refused - with the event log as evidence.

STEP 8 — Dispatch engine (the core edge — use Fable 5 here)

Goal: Geospatial offer→accept dispatch with atomic acceptance and honest failure paths. Where rivals fail.

Paste this prompt:

```
Read CLAUDE.md + master plan Section 8. This is the most failure-sensitive module in the platform - plan in detail first, including the failure paths. Implement:
1. On vendor acceptance of a DELIVERY order: find online+idle Movers near pickup (PostGIS ST_DWithin, indexed).
2. Score candidates: ETA (Maps interface distance matrix) + current load + acceptance history. Pure, unit-tested scoring function.
3. Offer->accept loop: offer top candidate -> 20s timeout -> next -> widen radius -> after N rounds, honest 'no riders available' to customer AND vendor (vendor can hold or cancel).
4. ATOMIC acceptance: two riders accepting simultaneously resolves to exactly one winner at the database level (compare-and-set / unique constraint - prove it with a concurrency test).
5. Accepted job locks; abandonment tracked and deprioritises the rider in future scoring.
6. Live tracking: rider app posts GPS on active jobs; customer subscribes (WebSocket) and sees movement + state changes.
```

Acceptance criteria (all must be true before commit):

- Concurrency test: 10 simultaneous accepts on one job -> exactly 1 winner, 9 clean rejections
- Timeout cascade proven by tests incl. the total-failure path
- Scoring function unit-tested in isolation
- Tracking updates end-to-end in a manual run
- Suite green

Test hardest:

- The no-acceptance path (THE worst early failure mode: food cooked, nobody coming)
- Riders going offline mid-offer; GPS dropouts mid-delivery; duplicate offer prevention

Done when

Dispatch survives a simulated rush hour with flaky riders, and every failure ends in an honest message - never a silent hang.

STEP 9 — Taxi vertical

Goal: Rides on the same mover pool and dispatch engine, with fixed upfront fares.

Paste this prompt:

```
Read CLAUDE.md + master plan Section 4.2. Plan first. Implement:  
1. RideRequest: pickup, dropoff, fixedFare computed and shown BEFORE  
   request; reuse dispatch (Step 8) for driver matching.  
2. Fare engine: zone-to-zone seeded table (admin-editable) with fallback  
   base + per-km + light per-minute. All rates in CountryConfig.  
3. Driver accept/decline; passenger sees driver identity, vehicle,  
   plate, live approach; post-trip rating both ways.  
4. Same atomicity + timeout guarantees as Step 8 (shared code, not a  
   copy).
```

Acceptance criteria (all must be true before commit):

- Fare is deterministic and shown before any driver sees the request
- Dispatch code is shared with delivery (no fork)
- Ratings recorded
- Suite green

Test hardest:

- Zone-boundary addresses; fare-table gaps falling back correctly

Done when

A passenger can request, see the exact fare, watch the car approach, and rate the trip - on the same engine that delivers food.

STEP 10 — Cash rules engine (golden rule, claims, strikes)

Goal: The Section 5 rules as code: claim flow, guarantee guardrails, strikes — rider trust depends on this.

Paste this prompt:

```
Read CLAUDE.md + master plan Section 5. Plan first. Implement:
1. Delivery handover screen enforces the golden rule: rider confirms
  'payment collected' to complete, OR flags 'no-show'/'refused' -
  available ONLY in the delivery state, capturing GPS, timestamp, and
  optional photo.
2. Failed-payment event -> ReimbursementClaim (if total under the $50-
  equivalent and golden rule followed) + Strike on customer account,
  phone, and address.
3. Guarantee guardrails: per-rider monthly claim cap (config); beyond
  cap -> review not auto-payout; claim-rate outlier flagging vs peer
  average; collusion detection (same customer/phone/address across
  multiple riders' claims, or one rider repeatedly vs one customer).
4. Strike consequences: thresholds (config) restrict ordering
  (verified-only -> ban). Level 3 Trusted accrual: completed paid
  orders + zero strikes + account age -> reduced friction flags.
5. Founder dashboard metrics: failed-payment % of orders, guarantee
  payout total/week, claims by rider.
```

Acceptance criteria (all must be true before commit):

- Claim impossible outside delivery state or without GPS stamp
- Caps, outlier and collusion flags proven by tests with synthetic patterns
- Strike thresholds restrict ordering as configured
- L3 promotion logic tested
- Suite green

Test hardest:

- A simulated dishonest rider (steady false claims) gets flagged; a simulated prankster customer gets restricted; an honest rider's clean claim pays

Done when

The whole Section 5 table of scenarios runs as an automated test suite, and the founder metrics update live.

STEP 11 — Notifications behind one interface

Goal: SMS, push, email unified — including THE loud vendor order alert.

Paste this prompt:

```
Read CLAUDE.md. Plan first. Implement:
1. Notifications interface: sms(), push(), email() - Twilio/FCM/SES-class
  adapters, swappable, with a dev/log adapter for local work.
2. Wire every event: OTPs, order status to customer, dispatch offers to
  riders, billing reminders/failures/suspension/reinstatement,
  verification outcomes, document-expiry warnings.
3. THE VENDOR ORDER ALERT: new order -> push + continuous audible ring
  on the vendor dashboard + persistent full-screen banner that cannot
  be dismissed until explicitly acknowledged (accept/reject/ack).
  Unacknowledged after N seconds -> re-alert + SMS fallback.
4. Per-user notification preferences where sane (the order alert is NOT
  optional for vendors).
```

Acceptance criteria (all must be true before commit):

- Every state change in the system emits through the interface (event audit test)
- Order alert: cannot be swiped away; re-alerts; SMS fallback fires
- Dev adapter logs everything locally
- Suite green

Test hardest:

- Vendor app backgrounded/locked when the order lands; notification permission denied paths

Done when

Kill the vendor app, place an order, and the phone still makes noise until someone acknowledges it.

STEP 12 — AI language layer (on top only)

Goal: One AiService for support and convenience — strictly off the money path.

Paste this prompt:

```
Read CLAUDE.md (rule 1 and 3 especially). Plan first. Implement:
1. One AiService wrapper around the Claude API. Uses ONLY: support chat,
  natural-language search intent -> structured filters ('cheap lunch
  near me open now'), nudge/notification phrasing, dispute thread
  summaries for admin, vendor menu-text polish.
2. Hard boundary in code review: AiService is never imported by
  dispatch, billing, auth, verification, or order-state modules.
  Add a lint rule/test enforcing the import boundary.
3. Async, off the critical path, graceful degradation when the API is
  down (the app must work fully without it).
4. No PII, documents, or payment data in any prompt - add a scrubber +
  test.
```

Acceptance criteria (all must be true before commit):

- Import-boundary check fails the build if violated
- App fully functional with AiService disabled
- Prompt scrubber proven by test
- Suite green

Test hardest:

- API-down behaviour; injection attempts via user text reaching tool prompts

Done when

Search understands 'cheap & close & open now', support chat works, and grep proves no money module touches AI.

STEP 13 — Dashboards (vendor, mover, admin)

Goal: The daily-use software surfaces — this is the B2B SaaS the subscription pays for.

Paste this prompt:

```
Read CLAUDE.md + master plan Section 7. Plan first. Implement:
1. VENDOR: live order queue (with the Step 11 alert), status actions,
  catalogue + availability toggles, subscription/tier/billing state,
  earnings record (their own, received directly), QR code download
  (their qrSlug as printable QR: 'scan to see our full catalogue').
2. MOVER: online/offline, offers with countdown, active job flow
  (pickup -> paid-vendor confirm -> navigate -> golden-rule handover),
  earnings, ratings, claims status, subscription state.
3. ADMIN (founder cockpit): verification review queue, suspensions,
  platform health, founder metrics (orders/day, active subscribers,
  failed-payment %, guarantee payouts, churn), support tooling.
4. Mobile-first; works on cheap Android devices; skeleton loaders;
  honest empty states.
```

Acceptance criteria (all must be true before commit):

- A vendor can run a full business day without leaving the dashboard
- A mover can run a full job incl. a failed handover claim
- Admin queue clears a verification end-to-end
- Suite green

Test hardest:

- Slow-network behaviour on the vendor order queue (it must degrade loudly, not silently)

Done when

You can play all three parts yourself - vendor, rider, admin - through one fake order with cash rules applied.

STEP 14 — Customer app polish + QR deep links

Goal: The consumer experience: fast, honest, trackable — and the QR acquisition loop.

Paste this prompt:

```
Read CLAUDE.md + master plan Section 9. Plan first. Implement:
1. QR deep links: /v/{qrSlug} resolves -> vendor page in-app; if app not
   installed -> store with deferred deep link to that vendor after
   install. This is the acquisition loop - test it on real devices.
2. Browse/search/discovery across vendor types; category UI driven by
   metadata; ad-boosted placement slots (clearly labelled).
3. Live order tracking map (Step 8 feed) that genuinely updates;
   skeleton loaders; optimistic UI; honest empty/error states
   ('no drivers nearby right now').
4. Saved addresses, reorder, order history. Totals always upfront.
5. Performance pass: cold start, image caching, low-end Android.
```

Acceptance criteria (all must be true before commit):

- QR scan -> vendor page works installed AND not-installed (deferred link)
- Tracking updates live in a manual two-phone test
- Boosted slots labelled
- Suite green

Test hardest:

- Deferred deep links on both iOS and Android (notoriously fiddly - test on hardware)

Done when

Scanning a printed QR on a phone without the app ends, two minutes later, with that vendor's catalogue open in the installed app.

STEP 15 — Hardening & observability

Goal: Production readiness: monitoring, load tests, backups, CI gates.

Paste this prompt:

```
Read CLAUDE.md. Plan first. Implement:
1. Sentry (or equiv) on backend + all apps; structured logging with a
   no-PII/no-secrets test on log output.
2. Load-test dispatch + order placement (simulated rush hour); fix what
   falls over; rate-limit public endpoints.
3. Backups + a TESTED restore (actually restore to a scratch DB in CI
   or a script you run - untested backups don't exist).
4. CI gates: typecheck + lint + tests + dependency/security scan must
   pass to merge. Dependabot + secret scanning on.
5. Runbook.md: deploy, rollback, restore, on-call basics.
```

Acceptance criteria (all must be true before commit):

- Restore proven, not assumed
- Load test results recorded with before/after fixes
- CI blocks a deliberately bad PR (test it)
- Suite green

Test hardest:

- Dispatch under load with slow DB responses; log-scrubbing under error storms

Done when

You could hand the runbook to a stranger and they could deploy, roll back, and restore.

STEP 16 — App store submission

Goal: iOS + Android live. Budget 1–2 weeks of review iterations — start while polishing.

Paste this prompt:

```
Read CLAUDE.md. Plan first. Prepare:  
1. Store listings (both stores): name, descriptions, screenshots on  
   required device sizes, category, age rating.  
2. Privacy 'nutrition' labels declaring exactly what's collected  
   (location during active orders, phone, verification docs for  
   workers) - consistent with the Privacy Policy.  
3. Live Privacy Policy URL + in-app account-deletion path (both stores  
   require them) - coordinate with the lawyer-reviewed documents.  
4. Release builds, signing, internal-test tracks first (TestFlight /  
   Play internal), then production review.  
5. A review-notes document for Apple/Google explaining the marketplace  
   model and demo credentials for a seeded test account of each role.
```

Acceptance criteria (all must be true before commit):

- Internal-test builds installable by you and 2-3 real testers
- Labels match the Privacy Policy exactly
- Demo accounts work for reviewers

Test hardest:

- Apple review edge: apps with worker onboarding/KYC often get questioned - the review notes preempt it

Done when

Both apps are in review or live, and a stranger can install from a store link, scan a QR, and order.

3. The 12-Day Fable Window (June 10–22)

Goal for the window: Steps 0–2 done properly, PR triage complete, and — if it's going well — the Step 8 dispatch skeleton, because that's the hardest module and Fable's biggest edge. NOT the whole platform: V1 is an 8–9 week build and that doesn't shrink because the model got faster. Steps below assume exams allow it; if they don't, compress to Steps 0–2 only — that's still the highest-value foundation.

Day	Work	Model
1	Create CLAUDE.md + docs/ in repo. Run Step 0 (orientation). Read ORIENTATION.md fully. Decide all 12 PRs.	Fable 5
2–3	Step 1: stabilise & secure. Don't rush; read SECURITY-FINDINGS.md.	Fable 5
4–6	Step 2: schema & multi-country + seed + smoke test + README realignment.	Fable 5
7	Buffer / spillover. Review everything committed so far end-to-end.	—
8–9	Step 3 (auth) — routine portions on Opus to save window quota.	Opus 4.8
10–12	Step 8 dispatch skeleton: scoring function, atomic acceptance, offer loop + concurrency tests — the hardest code in the platform, while Fable is cheap.	Fable 5

After the window (weeks 3–9, Opus-led)

Steps 4–7 (verification, billing, catalogue, orders) → finish 8, then 9–11 (taxi, cash rules, notifications) → 12–14 (AI layer, dashboards, customer polish + QR) → 15–16 (hardening, stores). Reach for Fable via credits only on steps marked 'test hardest'. In parallel, non-code: lawyer on the four documents, billing-rail confirmation in writing, KYC provider coverage check, IP assignment into the company.

4. The Non-Code Track (runs in parallel — founder only)

When	Action
Now	Assign IP into the Guyana company (one cheap legal document).
During build	Confirm recurring-billing rail for GYD subscription collection in writing. Verify KYC provider's Guyana ID coverage. Open/confirm the business bank account.
Before launch	Lawyer-review: Terms, Privacy Policy, Vendor Agreement, Mover Agreement. Insurance quotes (general liability + marketplace). Focused freelance security review of auth/billing paths (~\$1.5–5k).
Launch	Pick ONE Georgetown area. Recruit the vendor cluster (free first month). Print QR kits. Guarantee rider coverage at peak + late-night. Watch every early order personally.

The point of this playbook

The Master Plan answers 'what and why.' This volume answers 'what do I literally type next.' If you're ever unsure what to do, the answer is: the next unfinished step, top to bottom. No step is improved by another planning document — including this one. Day 1 starts with creating CLAUDE.md.

Companion to: Swift — The Master Plan (June 2026). Engineering guidance, not legal/financial advice.